

National Sun Yat-sen University
Department of Electrical Engineering
114-2 Practical Digital System Design

HW2 Voting and Median Circuit Design

班級：電機二乙

Name：周庭妤

Student ID：B133012015

一、 電路設計架構

➤ 平手規則(兩種電路皆相同)

在票數相同 (tie) 時，我設計於每一層比較中固定選擇右側輸入作為勝者，經由 comparator tree 累積後形成全域優先順序 ($10000 > 01000 > 00100 > 00010 > 00001$)；此機制可確保輸出結果唯一且可預測，且於兩種投票機架構中皆一致。

1. 投票機(Voting circuit)_第 1 種設計架構

本設計為一個 5-input voting circuit，其目的是從五組輸入 $a_0 \sim a_4$ 中，找出出現次數最多的 one-hot 候選值，並同時輸出對應的 one-hot 編碼以及其出現次數；整體架構可分為兩個主要部分：逐位計數 (bit-wise counting) 與比較樹 (comparator tree)。

(一) 逐位計數法 (Bit-wise Counting)

在第一階段中，電路針對每一個 bit (共 5 個 bit) 分別進行統計，也就是將 $a_0 \sim a_4$ 在同一個 bit 位置上的值加總，得到該 bit 上 1 出現的次數。

由於每次需要加總 5 個 1-bit 訊號，無法用單一元件完成，因此採用兩層 Full Adder + 一層 Half Adder 所構成的 adder tree 結構：先將三個輸入透過一個 FA 相加，其 sum 再與剩下兩個輸入透過第二個 FA 相加，最後二個 carry 用 HA 合併，最終輸出一個 3-bit 的計數結果(count)。

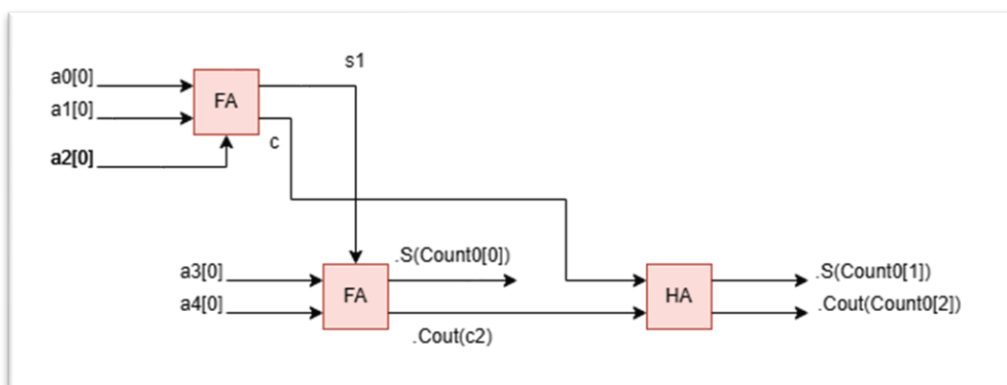


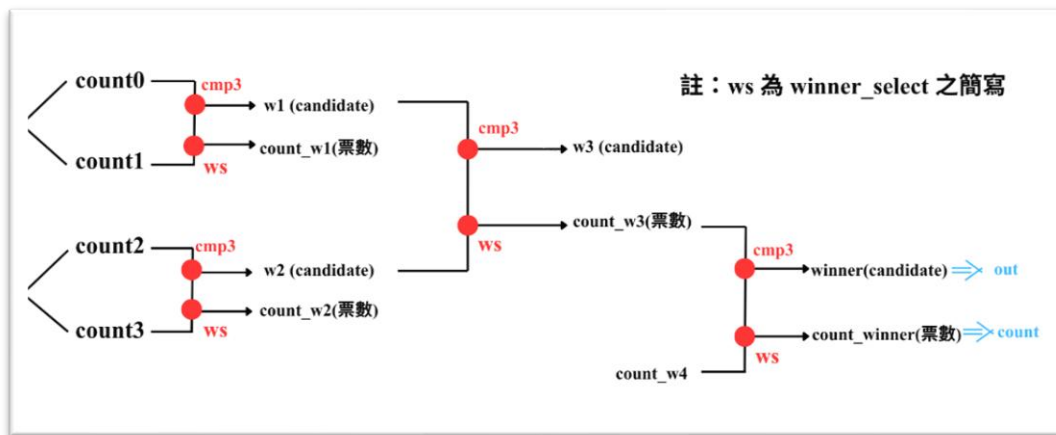
圖 (一)：bit-wise counting 內部架構
(以其中一組為例，此電路中共有五組此種架構)

(二) 比較樹 (Comparator Tree)

完成 count0~count4 後，接下來的目標是找出哪一個 count 最大，也就是哪個候選值得票最多；此架構整體分為三層：

- 第一層：count0 vs count1、count2 vs count3
- 第二層：前兩組勝者(w1,w2)再比較並得到 w3(指中間勝者 winner3)
- 第三層：w3 再與 count4 比較

每一次比較都透過 cmp3 判斷大小（產生 sel）以及 winner_select 同步選出較大 count + 對應 code



圖(二)：comparator tree 架構

(三) 整體系統架構 (System-Level View)

將前兩部分整合後，可以得到整體電路的整體架構：

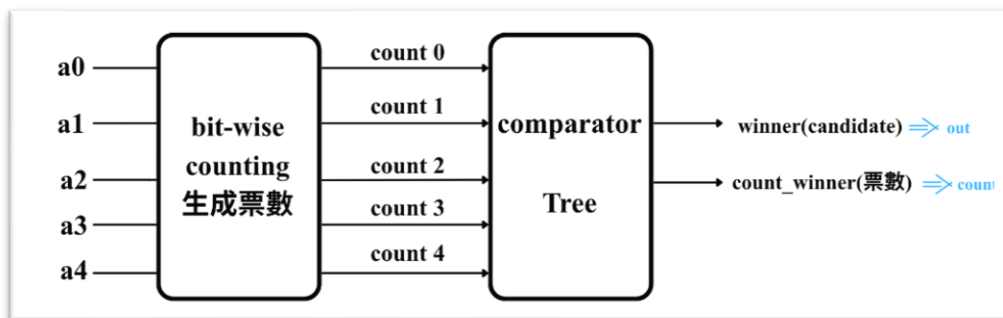


圖 (三)：voting circuit 總架構圖

詳見圖(三)；左側輸入為五組 one-hot 資料 a0~a4，首先進入 bit-wise counting 模組，針對每一個 bit 位置統計 1 出現的次數，並產生五個對應的計數結果 count0~count4；接著這些計數值會輸入 comparator tree 進行分層比較，逐步選出票數最大的候選值；最終輸出包含勝出選項的 one-hot 編碼 (out) 以及其對應的票數 (count)，完成整體投票判斷流程。

(四)架構總結分析

第一種投票機的設計採用逐位計數 (bit-wise counting) 搭配比較樹 (comparator tree) 的架構，其核心特性在於先將問題拆解為局部統計與全域決策兩個階段；在前段，透過 adder tree (由 Full Adder 與 Half Adder 組成) 對每一個 bit 獨立進行計數，將五組 one-hot 輸入轉換為五個 3-bit 的票數資訊 (count0~count4)；在後段，利用 comparator tree 進行階層式比較，並透過 winner_select 同步傳遞票數與對應候選位置，最終選出全域最大值。

此架構的優點在於：第一，模組化程度高，bit-wise counting 與 comparator tree 可獨立設計與驗證，電路結構清楚；第二，採用樹狀比較而非線性掃描，可有效縮短 critical path，提升速度表現；第三，透過結構化 (structural) 與資料流 (dataflow) 混合設計，使電路兼具硬體對應性與可讀性。

缺點則在於：硬體成本相對較高，因為每個 bit 都需要一組加法器樹，且 comparator tree 也需額外邏輯資源；此外，設計較偏向固定架構，若輸入數量擴展時，電路規模會快速增加。

在操作方法上，使用者輸入五組 one-hot 編碼資料 (a0~a4)，每一組代表一個候選選項，爾後電路會自動在各 bit 位置上統計 1 的出現次數，再透過比較樹找出出現最多次的 bit 位置，並輸出對應的 one-hot 結果 (out)，同時輸出其出現次數 (count)；當出現票數相同 (tie) 時，系統會依照預設優先順序 (10000 > 01000 > 00100 > 00010 > 00001) 決定最終輸出，確保結果唯一且穩定。

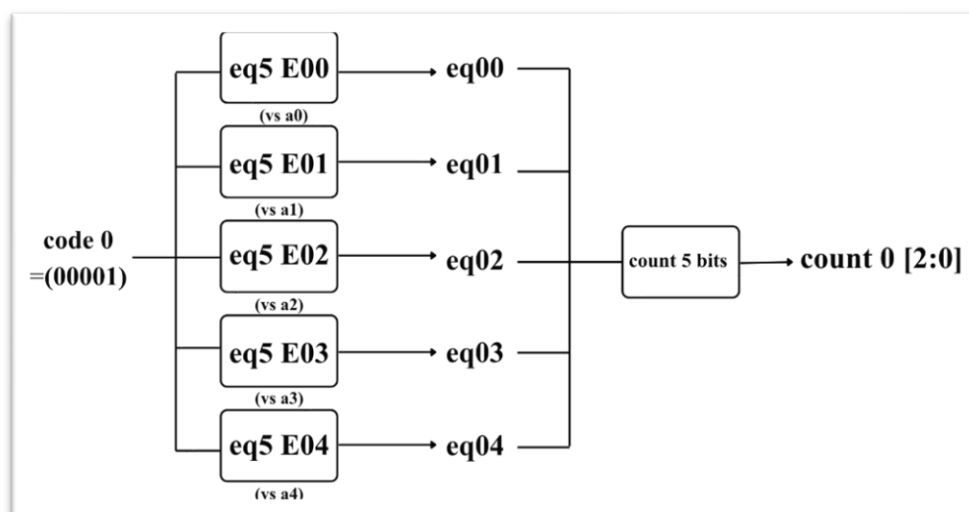
2. 投票機(Voting circuit)_第 2 種設計架構

此設計為一個 5-input voting circuit，其目的是從五組輸入 $a_0 \sim a_4$ 中，找出出現次數最多的 one-hot 候選值，並同時輸出對應的 one-hot 編碼與其票數；整體架構可分為兩個主要部分：候選比對計票 (Candidate Equality Matching) 與比較樹 (Comparator Tree)，作法為先把怎麼算票和怎麼選最大分成兩塊，再在 top module 做整合。

(一) 候選比對計票 (Candidate Equality Matching)

在第一階段中，電路不是直接把五組輸入彼此互相比較，而是先固定五個合法候選值，也就是 $code_0 \sim 4$ ；接著分別讓每一個候選值去和五組輸入 $a_0 \sim a_4$ 做 equality comparison，也就是說，對 $code_0$ 而言，會檢查它是否等於 a_0 、是否等於 a_1 、一路到是否等於 a_4 ，且對 $code_1 \sim code_4$ 都是做相同的事。

在硬體實現上，每個 equality comparison 由 eq5 模組負責；eq5 會逐 bit 比較兩個 5-bit one-hot 值是否完全相同，若全部相同則輸出 $eq=1$ ，接著，對於每一個候選值，會將它對五組輸入的比較結果共五個 1-bit 訊號送入 count5bits 模組；count5bits 內部再透過兩個 Full Adder 和一個 Half Adder，把五個 1-bit 結果加總成一個 3-bit 的票數結果，也就是說，此階段本質上是在做五個候選值 $\times$ 五組輸入的 matching，再把 matching 結果轉換成票數。



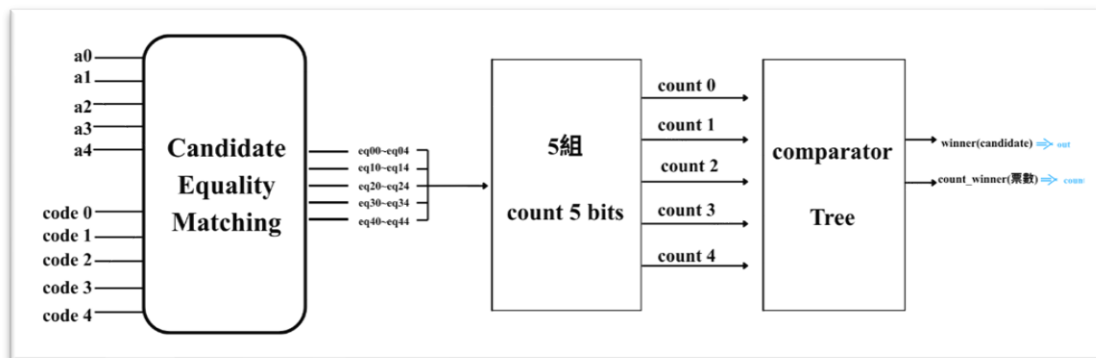
圖(四)：Candidate Equality Matching 內部架構
(以其中一組為例，此電路中共有五組此種架構)

(二) 比較樹 (Comparator Tree)

完成 count0~count4 後，接下來的目標就是找出哪一個 count 最大，也就是哪一個候選值得票最多；這部分架構其實和 voting(M1) 完全相同，仍然採用三層 comparator tree：第一層先比較 count0 vs count1 與 count2 vs count3；第二層再將前兩組勝者往上比較；第三層則將中間勝者再與 count4 比較，最後得到全域 winner(詳見圖(二)：comparator tree 架構)。

(三) 整體系統架構 (System-Level View)

將前兩部分整合後，可得整體投票機的完整架構；左側輸入為五組 one-hot 資料 a0~a4，以及五個固定候選值 code0~code4：系統先進入 Candidate Equality Matching 模組，逐一檢查每個候選值是否出現在輸入中，並產生五個對應的票數 count0~count4；接著這些票數再送入 Comparator Tree 進行分層比較，逐步選出最終勝出的候選值與其票數，最後分別輸出為 out 與 count，從整體資料流來看，整體電路結構為：先以 matching 方式計票，再以比較樹選出最大票數對應之候選值，作為最後贏家。



圖(五)：voting circuit 總架構圖

(四)架構總結分析

第二種投票機的設計採用候選比對計票 (candidate equality matching) 搭配比較樹 (comparator tree) 的架構，其核心特性在於先將問題轉換為對固定候選集合進行統計，再進行全域最大值選擇的兩階段處理；在前段，透過多組 equality comparator (eq5) 將每一個合法 one-hot 候選值 (00001~10000) 分別與五組輸入進行比對，並將比對結果送入計數模組 (由 Full Adder 與 Half Adder 組成的 count5bits)，將五組輸入轉換為五個 3-bit 的票數資訊 (count0~count4)；在後段，利用 comparator tree 進行階層式比較，並透過 winner_select 同步傳遞票數與對應候選位置，最終選出全域最大值。

此架構的優點在於：第一，計票方式直接對應候選值本身，邏輯語意清楚，相較 bit-wise counting 更直觀，較不易產生錯誤；第二，模組化程度高，equality comparison、count5bits 與 comparator tree 可各自獨立設計與驗證，提升設計與除錯效率；第三，透過比較樹進行非線性分層比較，可有效縮短 critical path，維持良好的速度表現；第四，結構上結合 structural 與 dataflow 設計，使電路同時具備硬體對應性與可讀性。

缺點則在於：硬體資源使用量較高，因為每一個候選值都需要對所有輸入進行完整比對，導致 equality comparator 的數量增加 (共 5×5 組)，再加上後續的加法器與 comparator tree，使整體面積成本上升；此外，設計同樣偏向固定架構，當輸入數量或候選數量擴展時，comparison 與計數模組會呈平方關係成長，電路規模增加更為明顯。

在操作方法上，使用者輸入五組 one-hot 編碼資料 (a0~a4)，電路首先針對五個合法候選值逐一檢查其是否出現在輸入中，並統計各候選值的出現次數；接著透過比較樹找出票數最大的候選值，並輸出對應的 one-hot 結果 (out) 與其票數 (count)；當出現票數相同 (tie) 時，系統會依照預設優先順序 (10000 > 01000 > 00100 > 00010 > 00001) 決定最終輸出，確保結果唯一且穩定。

3. 中位數機(Median circuit)

本設計為一個 5-input median selector，其目的是從五組輸入 $a_0 \sim a_4$ （皆為 6-bit one-hot 編碼）中，找出排序後位於中間的值（median），並輸出對應的 one-hot 編碼；整體設計的核心概念是將 one-hot 資料轉換為可比較的數值後，透過分階段的排序與範圍縮減來找出中位數，最後再轉回 one-hot 表示，整體架構可分為五個主要階段：編碼（encoding）、局部排序（local sorting）、範圍縮減（range reduction）、中位數選取（median selection）以及解碼（decoding）。

（一）編碼階段（Encoding）

在第一個階段中，電路透過 `encoder_6to3` 將每一組 6-bit one-hot 輸入轉換為對應的 3-bit index（0~5）；也就是將原本表示位置的 one-hot 資訊轉換為可以進行大小比較的數值形式。

（二）局部排序（Local Sorting）

接著在局部排序階段，電路將五個輸入分成兩組： (x_0, x_1) 與 (x_2, x_3) ，分別透過 `sort2` 模組進行排序，得到每一組的較小值（low）與較大值（high），而 `sort2` 的實作是基於 `cmp3` 比較器來判斷兩個 3-bit 數值的大小，並透過條件選擇（conditional operator）決定輸出順序，這個階段的目的是在於先在小範圍內建立基本的大小關係，避免直接對五個數進行完整排序，從而降低後續電路的複雜度與邏輯層數。

（三）範圍縮減（Range Reduction）

在完成局部排序後，電路進一步將兩組排序結果進行整合；具體做法是：取兩組 low 中的較大者作為下界 L （max of lows），以及兩組 high 中的較小者作為上界 H （min of highs）；這個步驟的核心觀念是：中位數一定會落在區間 $[L, H]$ 之間，因此可以有效縮小搜尋範圍，透過這樣的設計，就可以避免對五個數做完整排序（full sorting），改以較少的比較與邏輯資源來逼近答案，達到降低硬體成本與延遲的效果。

（四）中位數選取（Median Selection）

在範圍縮減後，剩下三個關鍵候選值： L 、 H 以及尚未處理的 x_4 ；此時電路將這三個數輸入 `median3` 模組，透過以下方式找出中位數：先對其中兩個數排序，再利用 `max2` 與 `min2` 的組合邏輯，求出三者中的中間值，這個設計本質

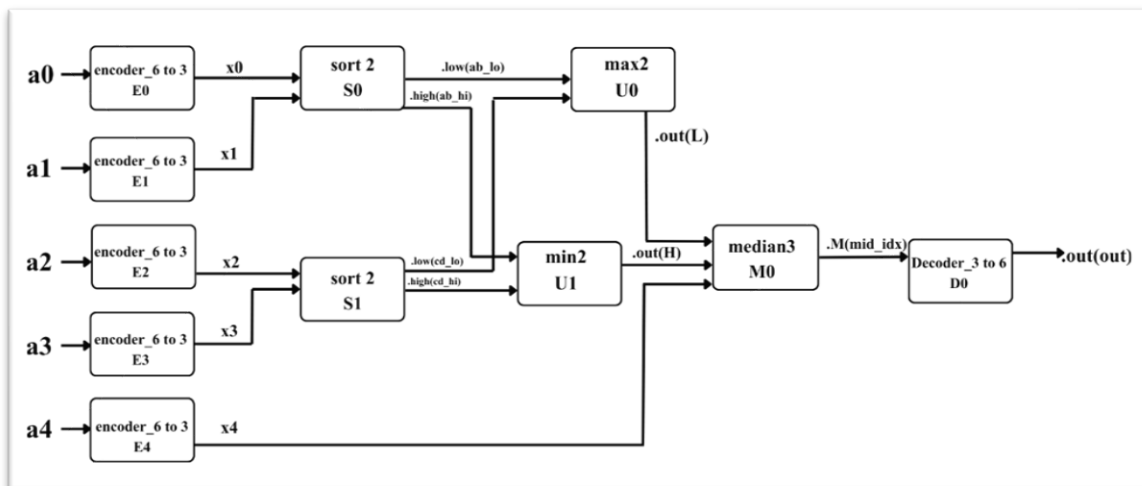
上是將 5-input median 問題轉換為 3-input median 問題，大幅簡化電路結構與比較次數，是整體設計中最關鍵的優化步驟。

(五) 解碼 (Decoding)

最後，在解碼階段，電路透過 decoder_3to6 將得到的 3-bit 中位數 index (mid_idx) 轉換回 6-bit one-hot 輸出；至此，整個資料流程完成：輸入的 one-hot 資料 → 轉為數值 → 經過排序與篩選 → 再轉回 one-hot 表示，確保輸出格式符合題目要求，同時保留原始候選位置資訊。

(六) 整體架構特性說明

在比較邏輯方面，cmp3 採用由高位元到低位元的優先比較方式 (MSB priority comparison)，確保 3-bit 數值比較的正確性；而 sort2、max2、min2 等模組皆建立在此比較器之上，形成良好的模組重用架構；整體電路在結構上採用結構化 (structural) 方式串接各子模組，並搭配資料流 (dataflow) 描述比較與選擇邏輯，是為混合式設計，使電路同時具備硬體對應性與良好可讀性；此外，相較於直接使用完整排序網路 (full sorting network)，我的設計透過局部排序 → 範圍縮減 → 三數中位數的策略，有效減少比較次數與邏輯層數，因此在 critical path (關鍵路徑) 與硬體成本上皆具有優勢，這也是我在架構選擇上的主要考量。



圖(六)：median circuit 總架構圖

 電路架構比較表

項目	Voting M1 (Bit-wise)	Voting M2 (Candidate)	Median Circuit
結構組成方式	bit-slice 平行結構	candidate-slice 平行結構	Partial sorting + Median selection
主要運算模組	Full Adder / Half Adder	comparator + adder	sort2 / min2 / max2 / median3
資料流型態	橫向 (bit-by-bit 累加)	縱向 (candidate-by-candidate)	分層串接 (stage-by-stage)
關鍵結構特徵	carry propagation chain	comparator cascade	comparator network
關鍵路徑來源	adder carry chain + comparator	comparator tree 深度	多層比較 + MUX 串接
結構深度 (邏輯層數)	中 (受 carry chain 影響)	高 (多層 comparator)	低~中 (固定層數)
邏輯閘分佈	集中於 adder	分散於 comparator	分布於多個小型模組
模組耦合程度	中 (adder 與 comparator 串接)	高 (多 comparator 相依)	低 (模組間較獨立)
可重複結構性	高 (bit-slice 可複製)	高 (candidate 可複製)	中 (需重新設計排序結構)

二、 演算法比較

1. 投票機 M1 (Bit-wise Counting) 演算法分析

第一種投票機的演算法核心是將原本的候選值投票問題轉換為位元出現次數統計問題，也就是將五組 one-hot 輸入在每一個 bit 位置上分開來看，分別統計該 bit 出現 1 的次數；具體做法是對每一個 bit (共 5 個) 建立一個加法樹 (adder tree)，透過 Full Adder 與 Half Adder 將五個 1-bit 訊號累加成一個 3-bit 的計數值 (count0~count4)，代表該 bit 被選中的次數，完成所有 bit 的統計後，再利用 comparator tree 進行階層式比較，逐步找出最大值，並同時傳遞對應的候選位置，最終得到票數最多的 one-hot 輸出；當出現平手情況時，則依照固定優先順序決定輸出，使結果唯一且穩定。

此演算法的特性在於將問題拆解為多個獨立的 bit 層級處理，具有良好的硬體對應性與規律結構，適合以結構化方式實現，且透過樹狀比較可以有效降低 critical path；然而其缺點在於需要為每一個 bit 建立一組加法器電路，使硬體資源消耗增加，且演算法本身較為抽象，需要從 bit 層級再回推至候選值，理解上相對不直觀。

2. 投票機 M2 (Candidate Matching) 演算法分析

第二種投票機的演算法則是直接從候選值本身出發，將問題轉換為每個候選值出現幾次的統計問題，整體流程為先定義所有可能的 one-hot 候選值，接著利用 equality comparator (eq5) 逐一比較每一個輸入是否等於該候選值，將比對結果轉為 1 或 0，再透過加法器將這些結果累加成該候選值的票數 (count0~count4)；完成所有候選值的計數後，同樣透過 comparator tree 進行階層式比較，找出票數最大的候選值並輸出。

此演算法的優點在於邏輯直觀，完全符合我們對投票的理解方式，且每個候選值的計算互相獨立，模組化程度高，易於設計與驗證；但其缺點在於硬體成本較高，因為需要對每一個候選值建立完整的比較與計數電路，導致比較器與加法器數量增加，且當候選數量擴展時，電路規模會快速成長。

在操作上，使用者輸入五組 one-hot 資料後，電路會逐一統計每個候選值的出現次數，再透過比較樹選出最大值，並依照既定優先順序處理平手情況。

3. 中位數機 (Median Circuit) 演算法分析

中位數機的演算法與投票機不同，其核心目標並非找最大值，而是找排序後位於中間的數值，因此設計上採用部分排序與範圍縮減的策略來避免完整排序。

首先將 one-hot 輸入透過 encoder 轉換為可比較的 binary index，接著進行局部排序（將五個數分組並排序），建立基本的大小關係，之後透過範圍縮減（range reduction）計算出一個必定包含中位數的區間 $[L, H]$ ，從而排除不可能的候選值，最後只需對剩餘的三個數（ L 、 H 、 x_4 ）進行 median-of-three 計算，即可得到最終的中位數，再透過 decoder 轉回 one-hot 輸出。

此演算法的優點在於大幅減少比較次數與邏輯層數，相較於完整排序網路能有效降低硬體成本並縮短 critical path，同時保有良好的計算效率，但其缺點在於設計流程較複雜，需要多個轉換階段（encoding 與 decoding），且演算法較不直觀，對設計者理解要求較高；在操作上，電路會先將輸入轉為數值，再透過排序與篩選逐步縮小範圍，最後輸出中位數對應的 one-hot 結果。

演算法分析比較表

項目	Voting M1(Bit-wise)	Voting M2 (Candidate)	Median Circuit
問題轉換方式	bit-level population count	candidate-wise counting + max selection	partial sorting + median selection
計算策略	每個 bit 獨立累加 (FA/HA)	每個 candidate 計數後比較大小	分階段排序 (sort2 / min / max)
計算效率特性	adder tree (carry propagation)	count + comparator tree	comparator network (固定深度)
演算法直觀性	加法累積 (較數學化)	比較樹 (較直觀)	排序與中值選取 (結構化)
核心優勢	結構規則、易擴展	邏輯直觀、易理解	critical path 較短、比較次數少
主要缺點	carry chain 導致延遲較大	gate count 高、硬體成本大	設計複雜、結構不直觀

三、 電路分析

這次我的設計之效能評估採用 EDA 工具 (Quartus Prime 20.1) 之編譯結果作為主要數據來源，理由在於：現代數位電路設計已由硬體描述語言 (HDL) 主導，編譯器在合成過程會進行複雜的邏輯優化 (如 Logic Minimization 與 Technology Mapping)，自動將冗餘邏輯合併並對應至硬體底層之自適應查找表 (ALUT)，因此，直接採用實作報告數據，能更真實且精確地反映電路在晶片上的空間佔用 (面積) 與訊號傳遞之時序表現 (延遲)，比傳統手動計算法更具實務參考價值。

基本邏輯閘等效數量與延遲分析單位表(以 2-input NAND 為基準)

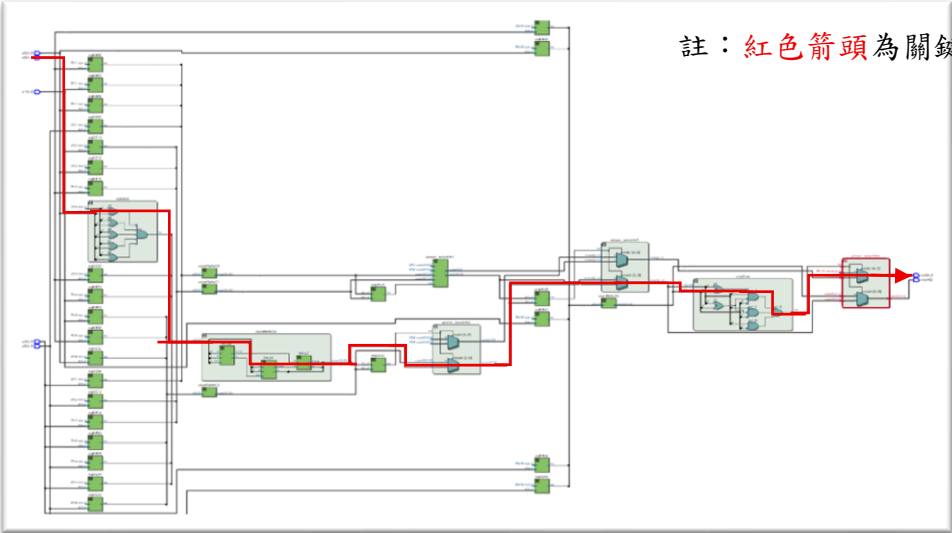
在數位電路設計中，為統一衡量標準，因此這裡設定一個 2-input NAND 為 1 個邏輯閘單位 (Gate Count = 1)，其傳遞延遲為 1 個單位延遲 ($T_{pd} = 1$)，其餘邏輯閘之等效數據推導如下：

Logic Gate	邏輯閘數量	等效延遲	推導說明 (NAND 實作邏輯)
NOT	1個	1T	將 NAND 的兩個輸入端相連
AND	2個	2T	一個 NAND 後接一個 NOT (1+1)
OR	3個	2T	根據笛摩根定律： $\overline{A \cdot B}$ 需 2 個 NOT 與 1 個 NAND
XOR	4個	3T	使用 4 個 NAND 構成，訊號最長經過 3 層閘
XNOR	5個	4T	一個 XOR 後接一個 NOT (3+1)

Voting circuit(M1)的電路分析

項目	Voting circuit(M1)																																																																																																																																				
Quartus 合成結果	註：紅色箭頭為關鍵路徑標示																																																																																																																																				
邏輯閘數量計算	根據設計之階層化結構統計，本電路（M1）之模組組成與等效個數換算如下： - Full Adder (FA)： 10 個 x 15 個 = 150 個 - Half Adder (HA)： 5 個 x 6 個 = 30 個 - Comparator (cmp3)： 4 個 x 15 個 = 60 個 - Winner_select (MUX 組)： 4 個 x 16 個 = 64 個 計算結果：本設計理論之總等效邏輯閘數量為： 150 + 30 + 60 + 64 = 304 個（以 NAND 為基準單位）。																																																																																																																																				
邏輯閘層數計算	經關鍵路徑追蹤，如圖紅線所示，訊號由輸入端 a1[0] 傳遞至輸出端 out[1]，依序經過第一層 Full Adder、第二層 Full Adder、Half Adder、三級 comparator 與三級 winner_select；若以 2-input NAND 等效方式估算其邏輯層數，則第一層 FA 約為 7 層、第二層 FA 約為 7 層、HA 約為 3 層、每級 cmp3 約為 8~10 層、每級 winner_select 約為 3~4 層，因此關鍵路徑之理論總等效邏輯閘層數約為： 7 + 7 + 3 + 8~10 + 3~4 + 8~10 + 3~4 + 8~10 + 3~4 = 50~59層																																																																																																																																				
關鍵路徑延遲計算	根據 Quartus Timing Analyzer 分析結果，本電路之最長延遲路徑（critical path）由輸入端 a1[0]傳遞至輸出端 out[1]，其 propagation delay 約為：$T_{pd} \approx 18.676 \text{ ns}$ 由報告可觀察，各輸入至輸出之延遲皆落於約 15 ns ~ 18 ns 範圍內，其中最大值即對應本電路之關鍵路徑，此路徑主要經過 bit-wise counting 之加法器結構與後段 comparator tree 與 winner_select 模組，顯示延遲主要累積於多層運算與比較邏輯的串接。 ←左圖為投票機(M1)關鍵路徑之時序報告 <table><tr><th colspan="6">Propagation Delay</th></tr><tr><th>Input Port</th><th>Output Port</th><th>RR</th><th>RF</th><th>FR</th><th>FF</th></tr><tr><td>181 a0[2]</td><td>out[1]</td><td>15.132</td><td>15.964</td><td>15.569</td><td>16.410</td></tr><tr><td>182 a1[1]</td><td>out[3]</td><td>15.308</td><td>15.892</td><td>15.855</td><td>16.411</td></tr><tr><td>183 a0[4]</td><td>out[1]</td><td>14.866</td><td>15.567</td><td>15.711</td><td>16.428</td></tr><tr><td>184 a2[2]</td><td>out[3]</td><td>15.059</td><td>15.715</td><td>15.883</td><td>16.548</td></tr><tr><td>185 a3[3]</td><td>out[1]</td><td>15.751</td><td>16.519</td><td>15.953</td><td>16.711</td></tr><tr><td>186 a3[2]</td><td>count[0]</td><td>15.192</td><td>15.950</td><td>15.978</td><td>16.745</td></tr><tr><td>187 a3[1]</td><td>out[0]</td><td>16.994</td><td>16.101</td><td>17.730</td><td>16.777</td></tr><tr><td>188 a4[0]</td><td>out[1]</td><td>15.949</td><td>16.791</td><td>16.015</td><td>16.869</td></tr><tr><td>189 a3[2]</td><td>out[3]</td><td>15.465</td><td>16.121</td><td>16.251</td><td>16.916</td></tr><tr><td>190 a1[1]</td><td>out[0]</td><td>17.281</td><td>16.445</td><td>17.828</td><td>16.964</td></tr><tr><td>191 a2[2]</td><td>out[0]</td><td>17.032</td><td>16.268</td><td>17.856</td><td>17.101</td></tr><tr><td>192 a1[0]</td><td>count[0]</td><td>15.573</td><td>16.359</td><td>16.336</td><td>17.122</td></tr><tr><td>193 a1[0]</td><td>out[3]</td><td>15.898</td><td>16.564</td><td>16.649</td><td>17.327</td></tr><tr><td>194 a3[2]</td><td>out[0]</td><td>17.438</td><td>16.674</td><td>18.224</td><td>17.469</td></tr><tr><td>195 a3[1]</td><td>out[1]</td><td>16.194</td><td>16.897</td><td>16.930</td><td>17.573</td></tr><tr><td>196 a1[1]</td><td>out[1]</td><td>16.481</td><td>17.241</td><td>17.028</td><td>17.760</td></tr><tr><td>197 a1[0]</td><td>out[0]</td><td>17.871</td><td>17.117</td><td>18.622</td><td>17.880</td></tr><tr><td>198 a2[2]</td><td>out[1]</td><td>16.232</td><td>17.064</td><td>17.056</td><td>17.897</td></tr><tr><td>199 a3[2]</td><td>out[1]</td><td>16.638</td><td>17.470</td><td>17.424</td><td>18.265</td></tr><tr><td>200 a1[0]</td><td>out[1]</td><td>17.071</td><td>17.913</td><td>17.822</td><td>18.676</td></tr></table>	Propagation Delay						Input Port	Output Port	RR	RF	FR	FF	181 a0[2]	out[1]	15.132	15.964	15.569	16.410	182 a1[1]	out[3]	15.308	15.892	15.855	16.411	183 a0[4]	out[1]	14.866	15.567	15.711	16.428	184 a2[2]	out[3]	15.059	15.715	15.883	16.548	185 a3[3]	out[1]	15.751	16.519	15.953	16.711	186 a3[2]	count[0]	15.192	15.950	15.978	16.745	187 a3[1]	out[0]	16.994	16.101	17.730	16.777	188 a4[0]	out[1]	15.949	16.791	16.015	16.869	189 a3[2]	out[3]	15.465	16.121	16.251	16.916	190 a1[1]	out[0]	17.281	16.445	17.828	16.964	191 a2[2]	out[0]	17.032	16.268	17.856	17.101	192 a1[0]	count[0]	15.573	16.359	16.336	17.122	193 a1[0]	out[3]	15.898	16.564	16.649	17.327	194 a3[2]	out[0]	17.438	16.674	18.224	17.469	195 a3[1]	out[1]	16.194	16.897	16.930	17.573	196 a1[1]	out[1]	16.481	17.241	17.028	17.760	197 a1[0]	out[0]	17.871	17.117	18.622	17.880	198 a2[2]	out[1]	16.232	17.064	17.056	17.897	199 a3[2]	out[1]	16.638	17.470	17.424	18.265	200 a1[0]	out[1]	17.071	17.913	17.822	18.676
Propagation Delay																																																																																																																																					
Input Port	Output Port	RR	RF	FR	FF																																																																																																																																
181 a0[2]	out[1]	15.132	15.964	15.569	16.410																																																																																																																																
182 a1[1]	out[3]	15.308	15.892	15.855	16.411																																																																																																																																
183 a0[4]	out[1]	14.866	15.567	15.711	16.428																																																																																																																																
184 a2[2]	out[3]	15.059	15.715	15.883	16.548																																																																																																																																
185 a3[3]	out[1]	15.751	16.519	15.953	16.711																																																																																																																																
186 a3[2]	count[0]	15.192	15.950	15.978	16.745																																																																																																																																
187 a3[1]	out[0]	16.994	16.101	17.730	16.777																																																																																																																																
188 a4[0]	out[1]	15.949	16.791	16.015	16.869																																																																																																																																
189 a3[2]	out[3]	15.465	16.121	16.251	16.916																																																																																																																																
190 a1[1]	out[0]	17.281	16.445	17.828	16.964																																																																																																																																
191 a2[2]	out[0]	17.032	16.268	17.856	17.101																																																																																																																																
192 a1[0]	count[0]	15.573	16.359	16.336	17.122																																																																																																																																
193 a1[0]	out[3]	15.898	16.564	16.649	17.327																																																																																																																																
194 a3[2]	out[0]	17.438	16.674	18.224	17.469																																																																																																																																
195 a3[1]	out[1]	16.194	16.897	16.930	17.573																																																																																																																																
196 a1[1]	out[1]	16.481	17.241	17.028	17.760																																																																																																																																
197 a1[0]	out[0]	17.871	17.117	18.622	17.880																																																																																																																																
198 a2[2]	out[1]	16.232	17.064	17.056	17.897																																																																																																																																
199 a3[2]	out[1]	16.638	17.470	17.424	18.265																																																																																																																																
200 a1[0]	out[1]	17.071	17.913	17.822	18.676																																																																																																																																

Voting circuit(M2)的電路分析

項目	Voting circuit(M2)																																																																																																																																																																			
Quartus 合成結果	 註：紅色箭頭為關鍵路徑標示																																																																																																																																																																			
邏輯閘數量計算	根據本電路之階層化結構統計，其模組組成為：eq5 共 25 個、count5bits 共 5 個、cmp3 共 4 個、winner_select 共 4 個；若以 2-input NAND 為基準換算，eq5 可視為 5 個 XNOR 與 4 個 AND，約為 $5 \times 5 + 4 \times 2 = 33$個；count5bits 由 2 個 FA 與 1 個 HA 組成，約為 $2 \times 15 + 6 = 36$個；cmp3 由 XNOR、NOT、AND 與 OR 所組成，約為 29 個；winner_select 為 8-bit 2:1 MUX，約為 64 個，因此本設計理論之總等效邏輯閘數量約為：$25 \times 33 + 5 \times 36 + 4 \times 29 + 4 \times 64 = 825 + 180 + 116 + 256 = 1377$，故可估計本電路之理論總等效邏輯閘數量約為 1377 個（以 NAND 為基準單位）。																																																																																																																																																																			
邏輯閘層數計算	經關鍵路徑追蹤（如圖紅線所示），訊號須依序經過a2[2] → eq5(E22) → count5bits(C2) → cmp3(n2) → Winner_select(w2) → cmp3(n3) → Winner_select(w3) → cmp3(n4) → Winner_select(w4) → out[3] 若以 2-input NAND 等效邏輯閘層數估算，則：eq5 ≈ 10層（1 個 XNOR 為 4 層，後接 3 層 AND tree，共 4 + 6 = 10層），count5bits ≈ 14層（第一層 FA 約 7 層、第二層 FA 約 4 層、HA 約 3 層），cmp3 ≈ 8~10層，winner_select ≈ 3~4層，而關鍵路徑共經過 1 個 eq5、1 個 count5bits、3 個 cmp3 與 3 個 winner_select，因此理論總等效邏輯閘層數約為：$10 + 14 + 8 \sim 10 + 3 \sim 4 + 8 \sim 10 + 3 \sim 4 + 8 \sim 10 + 3 \sim 4 = 57 \sim 60$層																																																																																																																																																																			
關鍵路徑延遲計算	Propagation Delay <table><tr><th>Input Port</th><th>Output Port</th><th>RR</th><th>RF</th><th>FR</th><th>FF</th></tr><tr><td>175 a4[4]</td><td>out[0]</td><td>16.937</td><td>16.661</td><td>17.608</td><td>17.300</td></tr><tr><td>176 a1[2]</td><td>out[3]</td><td>16.460</td><td>16.975</td><td>16.814</td><td>17.331</td></tr><tr><td>177 a0[0]</td><td>out[0]</td><td>16.961</td><td>16.545</td><td>17.700</td><td>17.372</td></tr><tr><td>178 a2[4]</td><td>out[1]</td><td>16.492</td><td>17.236</td><td>16.667</td><td>17.426</td></tr><tr><td>179 a3[4]</td><td>count[1]</td><td>16.459</td><td>17.193</td><td>16.837</td><td>17.452</td></tr><tr><td>180 a2[3]</td><td>count[1]</td><td>16.367</td><td>17.101</td><td>16.732</td><td>17.481</td></tr><tr><td>181 a2[3]</td><td>out[1]</td><td>16.400</td><td>17.144</td><td>16.765</td><td>17.524</td></tr><tr><td>182 a2[2]</td><td>out[2]</td><td>17.256</td><td>16.927</td><td>17.960</td><td>17.586</td></tr><tr><td>183 a0[3]</td><td>count[1]</td><td>16.239</td><td>16.973</td><td>16.872</td><td>17.621</td></tr><tr><td>184 a0[3]</td><td>out[1]</td><td>16.272</td><td>17.016</td><td>16.905</td><td>17.664</td></tr><tr><td>185 a0[2]</td><td>count[1]</td><td>15.983</td><td>16.763</td><td>17.092</td><td>17.707</td></tr><tr><td>186 a2[4]</td><td>out[3]</td><td>16.962</td><td>17.519</td><td>17.137</td><td>17.709</td></tr><tr><td>187 a4[2]</td><td>out[1]</td><td>15.995</td><td>16.806</td><td>16.913</td><td>17.733</td></tr><tr><td>188 a3[3]</td><td>out[3]</td><td>16.870</td><td>17.427</td><td>17.235</td><td>17.807</td></tr><tr><td>189 a2[2]</td><td>out[0]</td><td>17.501</td><td>17.202</td><td>18.205</td><td>17.861</td></tr><tr><td>190 a0[1]</td><td>out[1]</td><td>16.419</td><td>17.230</td><td>17.050</td><td>17.869</td></tr><tr><td>191 a0[0]</td><td>count[1]</td><td>16.410</td><td>17.071</td><td>17.149</td><td>17.886</td></tr><tr><td>192 a4[4]</td><td>count[1]</td><td>16.464</td><td>17.187</td><td>17.250</td><td>17.907</td></tr><tr><td>193 a0[0]</td><td>out[1]</td><td>16.443</td><td>17.114</td><td>17.182</td><td>17.941</td></tr><tr><td>194 a0[3]</td><td>out[2]</td><td>16.742</td><td>17.299</td><td>17.375</td><td>17.947</td></tr><tr><td>195 a1[2]</td><td>out[3]</td><td>16.465</td><td>17.089</td><td>17.383</td><td>18.016</td></tr><tr><td>196 a4[4]</td><td>out[3]</td><td>16.889</td><td>17.513</td><td>17.560</td><td>18.152</td></tr><tr><td>197 a0[0]</td><td>out[3]</td><td>16.913</td><td>17.397</td><td>17.650</td><td>18.224</td></tr><tr><td>198 a2[2]</td><td>out[1]</td><td>16.983</td><td>17.771</td><td>17.687</td><td>18.430</td></tr><tr><td>199 a2[2]</td><td>count[1]</td><td>17.121</td><td>17.728</td><td>17.889</td><td>18.504</td></tr><tr><td>200 a2[2]</td><td>out[3]</td><td>17.453</td><td>18.054</td><td>18.157</td><td>18.713</td></tr></table>	Input Port	Output Port	RR	RF	FR	FF	175 a4[4]	out[0]	16.937	16.661	17.608	17.300	176 a1[2]	out[3]	16.460	16.975	16.814	17.331	177 a0[0]	out[0]	16.961	16.545	17.700	17.372	178 a2[4]	out[1]	16.492	17.236	16.667	17.426	179 a3[4]	count[1]	16.459	17.193	16.837	17.452	180 a2[3]	count[1]	16.367	17.101	16.732	17.481	181 a2[3]	out[1]	16.400	17.144	16.765	17.524	182 a2[2]	out[2]	17.256	16.927	17.960	17.586	183 a0[3]	count[1]	16.239	16.973	16.872	17.621	184 a0[3]	out[1]	16.272	17.016	16.905	17.664	185 a0[2]	count[1]	15.983	16.763	17.092	17.707	186 a2[4]	out[3]	16.962	17.519	17.137	17.709	187 a4[2]	out[1]	15.995	16.806	16.913	17.733	188 a3[3]	out[3]	16.870	17.427	17.235	17.807	189 a2[2]	out[0]	17.501	17.202	18.205	17.861	190 a0[1]	out[1]	16.419	17.230	17.050	17.869	191 a0[0]	count[1]	16.410	17.071	17.149	17.886	192 a4[4]	count[1]	16.464	17.187	17.250	17.907	193 a0[0]	out[1]	16.443	17.114	17.182	17.941	194 a0[3]	out[2]	16.742	17.299	17.375	17.947	195 a1[2]	out[3]	16.465	17.089	17.383	18.016	196 a4[4]	out[3]	16.889	17.513	17.560	18.152	197 a0[0]	out[3]	16.913	17.397	17.650	18.224	198 a2[2]	out[1]	16.983	17.771	17.687	18.430	199 a2[2]	count[1]	17.121	17.728	17.889	18.504	200 a2[2]	out[3]	17.453	18.054	18.157	18.713	根據 Quartus 的分析結果，本設計的關鍵路徑（Critical Path）是從輸入 a2[2] 傳到輸出 out[3]，其最大傳播延遲為 18.713 ns，實際觀察這條路徑，可以發現訊號會先經過 eq5 模組進行候選比較，再進入 count5bits 做票數統計，最後通過三層 comparator tree（包含 cmp3 與 winner_select）逐層選出最終勝者。由於這條路徑同時包含比較、加法與多層選擇邏輯，因此會形成整個電路中延遲最大的路徑。 ◀ 左圖為投票機(M2)關鍵路徑之時序報告
Input Port	Output Port	RR	RF	FR	FF																																																																																																																																																															
175 a4[4]	out[0]	16.937	16.661	17.608	17.300																																																																																																																																																															
176 a1[2]	out[3]	16.460	16.975	16.814	17.331																																																																																																																																																															
177 a0[0]	out[0]	16.961	16.545	17.700	17.372																																																																																																																																																															
178 a2[4]	out[1]	16.492	17.236	16.667	17.426																																																																																																																																																															
179 a3[4]	count[1]	16.459	17.193	16.837	17.452																																																																																																																																																															
180 a2[3]	count[1]	16.367	17.101	16.732	17.481																																																																																																																																																															
181 a2[3]	out[1]	16.400	17.144	16.765	17.524																																																																																																																																																															
182 a2[2]	out[2]	17.256	16.927	17.960	17.586																																																																																																																																																															
183 a0[3]	count[1]	16.239	16.973	16.872	17.621																																																																																																																																																															
184 a0[3]	out[1]	16.272	17.016	16.905	17.664																																																																																																																																																															
185 a0[2]	count[1]	15.983	16.763	17.092	17.707																																																																																																																																																															
186 a2[4]	out[3]	16.962	17.519	17.137	17.709																																																																																																																																																															
187 a4[2]	out[1]	15.995	16.806	16.913	17.733																																																																																																																																																															
188 a3[3]	out[3]	16.870	17.427	17.235	17.807																																																																																																																																																															
189 a2[2]	out[0]	17.501	17.202	18.205	17.861																																																																																																																																																															
190 a0[1]	out[1]	16.419	17.230	17.050	17.869																																																																																																																																																															
191 a0[0]	count[1]	16.410	17.071	17.149	17.886																																																																																																																																																															
192 a4[4]	count[1]	16.464	17.187	17.250	17.907																																																																																																																																																															
193 a0[0]	out[1]	16.443	17.114	17.182	17.941																																																																																																																																																															
194 a0[3]	out[2]	16.742	17.299	17.375	17.947																																																																																																																																																															
195 a1[2]	out[3]	16.465	17.089	17.383	18.016																																																																																																																																																															
196 a4[4]	out[3]	16.889	17.513	17.560	18.152																																																																																																																																																															
197 a0[0]	out[3]	16.913	17.397	17.650	18.224																																																																																																																																																															
198 a2[2]	out[1]	16.983	17.771	17.687	18.430																																																																																																																																																															
199 a2[2]	count[1]	17.121	17.728	17.889	18.504																																																																																																																																																															
200 a2[2]	out[3]	17.453	18.054	18.157	18.713																																																																																																																																																															

Median circuit 的電路分析

項目	Median circuit																																																																																																																																																																								
Quartus 合成結果	註：紅色箭頭為關鍵路徑標示																																																																																																																																																																								
邏輯閘數量計算	此電路階層包含 5 個 encoder_6to3、2 個 sort2、1 個 max2、1 個 min2、1 個 median3 及 1 個 decoder_3to6，依據基準單位換算，主要模組門數估計如下： • encoder/decoder：約 25 門 x 6 = 150 個 • sort/max/min (比較與交換邏輯)：約 35 個 x 4 = 140 個 • median3 (核心排序)：約 60 個 計算結果：本電路之理論總等效邏輯閘數量約為 350 個（以 NAND 為基準單位）。																																																																																																																																																																								
邏輯閘層數計算	經關鍵路徑追蹤（如圖紅線所示），訊號由輸入端 a3[5] 傳遞至輸出端 out[5]，依序經過 encoder_6to3(E3)、sort2(S1)、min2(U1)、median3(M0) 內部之 sort2(u0) 與 max2(u1)，最後進入 decoder_3to6(D0) 輸出。若以 2-input NAND 等效邏輯閘層數估算，則： encoder_6to3(E3) 約為 5 層，sort2(S1) 約為 10~12 層（cmp3 約 8~10 層，後接 2:1 MUX 約 2 層）min2(U1) 約為 10~12 層（cmp3 約 8~10 層，後接 2:1 MUX 約 2 層），median3(M0) 內部關鍵路徑約為 20~24 層，（其中 sort2(u0) 約 12 層，max2(u1) 約 12 層），decoder_3to6(D0) 約為 5 層 關鍵路徑之理論總等效邏輯閘層數約為： 5 + 10~12 + 10~12 + 20~24 + 5 = 50~58層																																																																																																																																																																								
關鍵路徑延遲計算	<table><tr><th colspan="6">Propagation Delay</th></tr><tr><th>Input Port</th><th>Output Port</th><th>RR</th><th>RF</th><th>FR</th><th>FF</th></tr><tr><td>125 a0[5]</td><td>out[1]</td><td>16.012</td><td>16.783</td><td>16.240</td><td>17.011</td></tr><tr><td>126 a0[3]</td><td>out[5]</td><td>16.256</td><td>17.159</td><td>16.163</td><td>17.057</td></tr><tr><td>127 a1[4]</td><td>out[1]</td><td>16.111</td><td>16.882</td><td>16.331</td><td>17.102</td></tr><tr><td>128 a1[5]</td><td>out[1]</td><td>16.107</td><td>16.878</td><td>16.338</td><td>17.109</td></tr><tr><td>129 a0[5]</td><td>out[3]</td><td>16.225</td><td>16.886</td><td>16.453</td><td>17.114</td></tr><tr><td>130 a1[3]</td><td>out[5]</td><td>16.402</td><td>17.305</td><td>16.222</td><td>17.116</td></tr><tr><td>131 a1[4]</td><td>out[3]</td><td>16.324</td><td>16.985</td><td>16.544</td><td>17.205</td></tr><tr><td>132 a3[3]</td><td>out[2]</td><td>16.111</td><td>16.639</td><td>16.698</td><td>17.209</td></tr><tr><td>133 a1[5]</td><td>out[3]</td><td>16.320</td><td>16.981</td><td>16.551</td><td>17.212</td></tr><tr><td>134 a3[3]</td><td>out[0]</td><td>17.450</td><td>16.824</td><td>18.037</td><td>17.394</td></tr><tr><td>135 a0[5]</td><td>out[5]</td><td>16.375</td><td>17.278</td><td>16.603</td><td>17.506</td></tr><tr><td>136 a3[5]</td><td>out[2]</td><td>16.193</td><td>16.721</td><td>17.024</td><td>17.552</td></tr><tr><td>137 a1[4]</td><td>out[5]</td><td>16.474</td><td>17.377</td><td>16.694</td><td>17.597</td></tr><tr><td>138 a1[5]</td><td>out[5]</td><td>16.470</td><td>17.373</td><td>16.701</td><td>17.604</td></tr><tr><td>139 a3[5]</td><td>out[0]</td><td>17.532</td><td>16.906</td><td>18.363</td><td>17.737</td></tr><tr><td>140 a3[3]</td><td>out[4]</td><td>16.682</td><td>17.500</td><td>17.269</td><td>18.070</td></tr><tr><td>141 a3[3]</td><td>out[1]</td><td>16.945</td><td>17.716</td><td>17.515</td><td>18.303</td></tr><tr><td>142 u3[5]</td><td>out[4]</td><td>16.764</td><td>17.582</td><td>17.595</td><td>18.413</td></tr><tr><td>143 a1[1]</td><td>out[1]</td><td>16.779</td><td></td><td></td><td>18.459</td></tr><tr><td>144 a1[1]</td><td>out[3]</td><td>16.863</td><td></td><td></td><td>18.562</td></tr><tr><td>145 a3[3]</td><td>out[3]</td><td>17.158</td><td>17.819</td><td>17.599</td><td>18.619</td></tr><tr><td>146 a3[5]</td><td>out[1]</td><td>17.027</td><td>17.798</td><td>17.858</td><td>18.629</td></tr><tr><td>147 a1[1]</td><td>out[5]</td><td>17.005</td><td></td><td></td><td>18.769</td></tr><tr><td>148 a3[5]</td><td>out[3]</td><td>17.240</td><td>17.976</td><td>18.071</td><td>18.821</td></tr><tr><td>149 a3[3]</td><td>out[5]</td><td>17.308</td><td>18.211</td><td>17.940</td><td>18.834</td></tr><tr><td>150 a3[5]</td><td>out[5]</td><td>17.390</td><td>18.293</td><td>18.221</td><td>19.124</td></tr></table> 根據 Quartus 的分析結果，關鍵路徑（Critical Path）是從輸入 a3[5] 傳到輸出 out[5]，其最大傳播延遲為 19.124 ns，實際觀察這條路徑，可以發現訊號會先經過 encoder_6to3 模組進行編碼，再依序進入 sort2 → min2 → median3 進行找出中位數的邏輯處理，此時狀態還是中位數對應之編碼，最後通過 decoder 將編碼轉換回相對應初始之輸入值找出中位數，由於該路徑同時串接最多層之比較與選擇邏輯，累積之組合延遲最大，因此會形成整個電路中延遲最大的路徑。 ←左圖為中位數機關鍵路徑之時序報告	Propagation Delay						Input Port	Output Port	RR	RF	FR	FF	125 a0[5]	out[1]	16.012	16.783	16.240	17.011	126 a0[3]	out[5]	16.256	17.159	16.163	17.057	127 a1[4]	out[1]	16.111	16.882	16.331	17.102	128 a1[5]	out[1]	16.107	16.878	16.338	17.109	129 a0[5]	out[3]	16.225	16.886	16.453	17.114	130 a1[3]	out[5]	16.402	17.305	16.222	17.116	131 a1[4]	out[3]	16.324	16.985	16.544	17.205	132 a3[3]	out[2]	16.111	16.639	16.698	17.209	133 a1[5]	out[3]	16.320	16.981	16.551	17.212	134 a3[3]	out[0]	17.450	16.824	18.037	17.394	135 a0[5]	out[5]	16.375	17.278	16.603	17.506	136 a3[5]	out[2]	16.193	16.721	17.024	17.552	137 a1[4]	out[5]	16.474	17.377	16.694	17.597	138 a1[5]	out[5]	16.470	17.373	16.701	17.604	139 a3[5]	out[0]	17.532	16.906	18.363	17.737	140 a3[3]	out[4]	16.682	17.500	17.269	18.070	141 a3[3]	out[1]	16.945	17.716	17.515	18.303	142 u3[5]	out[4]	16.764	17.582	17.595	18.413	143 a1[1]	out[1]	16.779			18.459	144 a1[1]	out[3]	16.863			18.562	145 a3[3]	out[3]	17.158	17.819	17.599	18.619	146 a3[5]	out[1]	17.027	17.798	17.858	18.629	147 a1[1]	out[5]	17.005			18.769	148 a3[5]	out[3]	17.240	17.976	18.071	18.821	149 a3[3]	out[5]	17.308	18.211	17.940	18.834	150 a3[5]	out[5]	17.390	18.293	18.221	19.124
Propagation Delay																																																																																																																																																																									
Input Port	Output Port	RR	RF	FR	FF																																																																																																																																																																				
125 a0[5]	out[1]	16.012	16.783	16.240	17.011																																																																																																																																																																				
126 a0[3]	out[5]	16.256	17.159	16.163	17.057																																																																																																																																																																				
127 a1[4]	out[1]	16.111	16.882	16.331	17.102																																																																																																																																																																				
128 a1[5]	out[1]	16.107	16.878	16.338	17.109																																																																																																																																																																				
129 a0[5]	out[3]	16.225	16.886	16.453	17.114																																																																																																																																																																				
130 a1[3]	out[5]	16.402	17.305	16.222	17.116																																																																																																																																																																				
131 a1[4]	out[3]	16.324	16.985	16.544	17.205																																																																																																																																																																				
132 a3[3]	out[2]	16.111	16.639	16.698	17.209																																																																																																																																																																				
133 a1[5]	out[3]	16.320	16.981	16.551	17.212																																																																																																																																																																				
134 a3[3]	out[0]	17.450	16.824	18.037	17.394																																																																																																																																																																				
135 a0[5]	out[5]	16.375	17.278	16.603	17.506																																																																																																																																																																				
136 a3[5]	out[2]	16.193	16.721	17.024	17.552																																																																																																																																																																				
137 a1[4]	out[5]	16.474	17.377	16.694	17.597																																																																																																																																																																				
138 a1[5]	out[5]	16.470	17.373	16.701	17.604																																																																																																																																																																				
139 a3[5]	out[0]	17.532	16.906	18.363	17.737																																																																																																																																																																				
140 a3[3]	out[4]	16.682	17.500	17.269	18.070																																																																																																																																																																				
141 a3[3]	out[1]	16.945	17.716	17.515	18.303																																																																																																																																																																				
142 u3[5]	out[4]	16.764	17.582	17.595	18.413																																																																																																																																																																				
143 a1[1]	out[1]	16.779			18.459																																																																																																																																																																				
144 a1[1]	out[3]	16.863			18.562																																																																																																																																																																				
145 a3[3]	out[3]	17.158	17.819	17.599	18.619																																																																																																																																																																				
146 a3[5]	out[1]	17.027	17.798	17.858	18.629																																																																																																																																																																				
147 a1[1]	out[5]	17.005			18.769																																																																																																																																																																				
148 a3[5]	out[3]	17.240	17.976	18.071	18.821																																																																																																																																																																				
149 a3[3]	out[5]	17.308	18.211	17.940	18.834																																																																																																																																																																				
150 a3[5]	out[5]	17.390	18.293	18.221	19.124																																																																																																																																																																				

四、測試平台與驗證完整性

(一) Testbench 設計理念

1. 投票機(Voting circuit)的 testbench

本次設計之 testbench 採用結合 corner cases 與完整測試 (exhaustive test) 的驗證策略，以確保電路在各種情況下皆能正確運作；在設計上，首先針對投票機容易出錯的關鍵情況（如多數決、平手情況、極端分布等）特別設計 corner cases，接著定義所有合法的 one-hot 輸入組合，並透過巢狀迴圈自動產生所有可能的測試情境，以確保測試覆蓋率的完整性。

在驗證方式上，testbench 內部建立對應的 golden model，即透過行為描述直接計算預期輸出 (expected output)，並與 DUT (Design Under Test) 進行逐筆比對，當結果不一致時，即輸出詳細錯誤訊息（包含輸入、DUT 輸出與預期結果），以利除錯；此外，透過計數器統計 pass 與 fail 次數，並在模擬結束時輸出總結結果，使整體驗證流程具備自動化與完整性。

2. 中位數機(Median circuit)的 testbench

中位數機的 testbench 同樣採用 corner cases 搭配完整測試 (exhaustive test) 的驗證方式，但在驗證重點上更著重於數值排序與中位數選取的正确性；由於 median 問題本質為排序問題，testbench 中的 golden model 會先將輸入轉換為對應的數值，再透過排序或條件判斷找出中位數，最後轉回 one-hot 作為預期輸出。

在測試資料設計上，除了涵蓋所有合法輸入組合外，也特別關注幾種關鍵情況，例如數值已排序、完全亂序、集中分布以及邊界值情況，以確保在不同排列下電路皆能正確找出中位數；驗證過程同樣透過自動比對 DUT 與 golden model 的結果來完成，並輸出錯誤案例與統計結果，使整體測試具備高覆蓋率與可靠性。

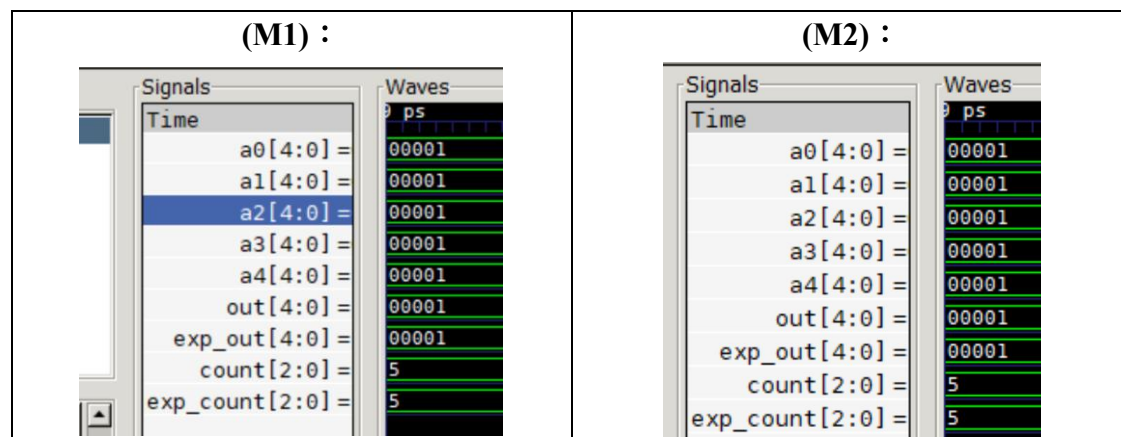
(二) 驗證完整性

1. 投票機(Voting circuit)的結果分析

本節針對幾種代表性 corner cases 進行波形觀察，並同時比較兩種投票機 (M1 與 M2) 之輸出結果，以驗證其功能正確性與平手處理一致性。

Case 1 : All Same

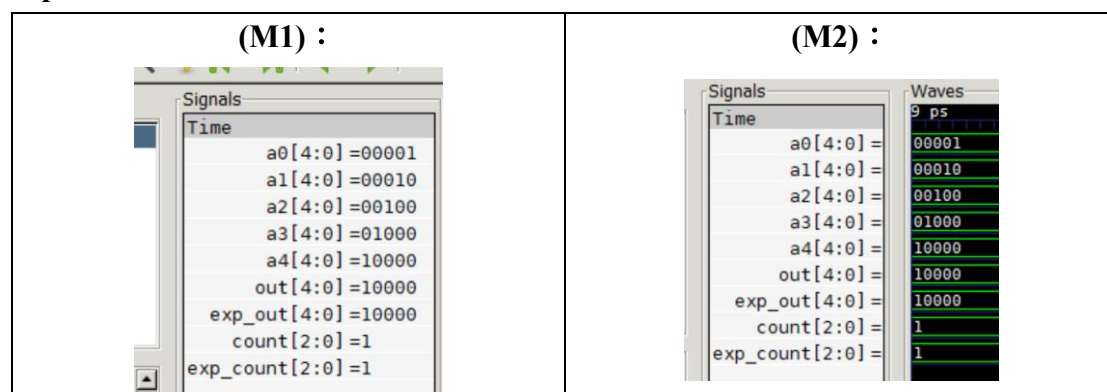
Input : $a_0 = 00001$, $a_1 = 00001$, $a_2 = 00001$, $a_3 = 00001$, $a_4 = 00001$



此案例為所有輸入皆相同之情況，所有候選者對應至同一個 one-hot 值 (00001)，因此其票數為 5。由於不存在不同候選之間的競爭，電路直接輸出該唯一候選，結果為 $out = 00001$, $count = 5$ ，與預期一致，顯示電路在單一候選情況下能正確運作。

Case 2 : All Different

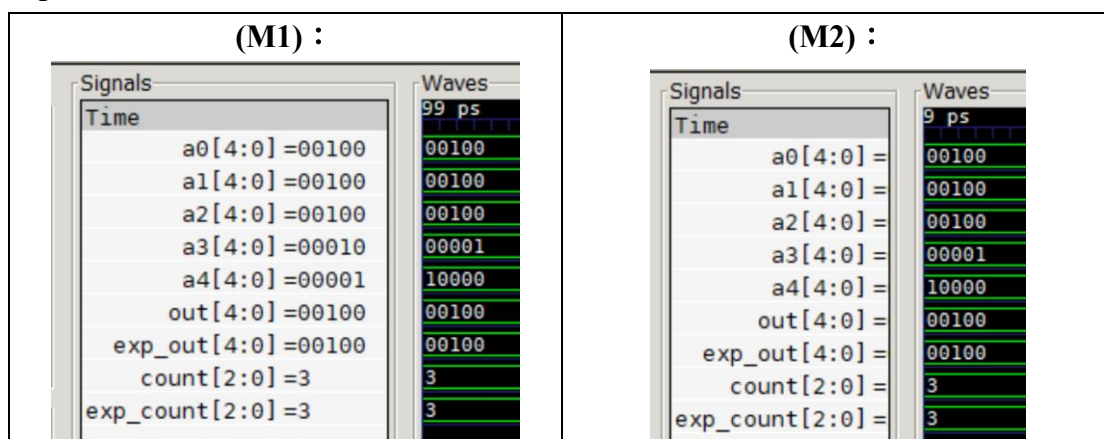
Input : $a_0 = 00001$, $a_1 = 00010$, $a_2 = 00100$, $a_3 = 01000$, $a_4 = 10000$



此案例為所有輸入皆不同之情況，每個候選者票數皆為 1，形成完全平手 (tie)；根據本設計之比較規則，在每一層比較中若票數相同則選擇右側輸入，經由 comparator tree 層層傳遞後，最終保留下來的是最右側候選 (10000)，因此輸出為 $out = 10000$, $count = 1$ (in decimal)，符合設計之平手處理邏輯。

Case 3 : 3-1-1

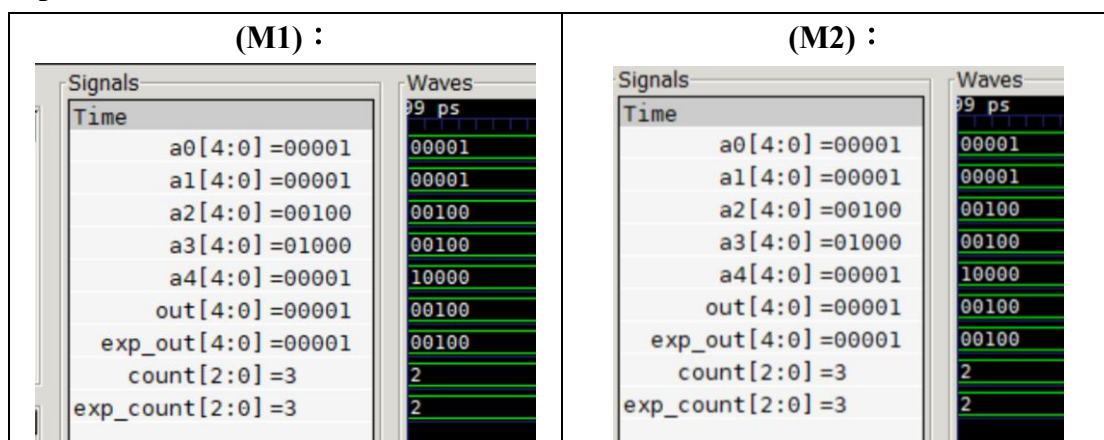
Input : a0 = 00100 , a1 = 00100 , a2 = 00100 , a3 = 00001 , a4 = 10000



此案例為明顯多數決情況，其中候選 00100 出現三次，其餘候選各出現一次。由於 00100 的票數明顯大於其他候選，因此在比較過程中不涉及平手判斷，電路可直接選出該候選，最終輸出為 out = 00100，count = 3(in decimal)，顯示電路能正確完成多數判斷功能。

Case 4 : 2-2-1 (Tie 關鍵)

Input : a0 = 00001 , a1 = 00001 , a2 = 00100 , a3 = 00100 , a4 = 01000



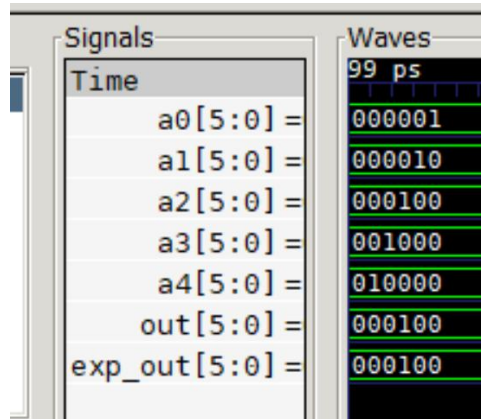
此案例為 2-2-1 情況，其中 00001 與 00100 皆出現兩次，形成平手，根據本設計之規則，在比較過程中若票數相同則選擇右側輸入，因此在比較 00001 與 00100 時，會保留右側之 00100，經由 comparator tree 傳遞後，最終輸出為 out = 00100，count = 2 (in decimal)，結果符合平手時偏向右側候選之設計邏輯。

[illegible]

2. 中位數機(Median circuit)的結果分析

Case 1：已排序輸入 (Already Sorted Case)

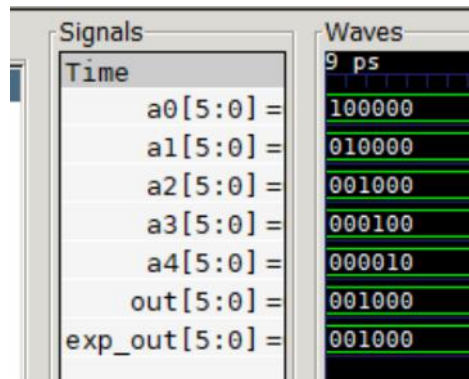
Input： a0 = 000001，a1 = 000010，a2 = 000100，a3 = 001000，a4 = 010000



此案例為輸入資料已依大小排序的情況，五個值由小到大依序排列，因此中位數應直接落在第三個位置。模擬結果顯示輸出為 out = 000100，符合預期，表示電路在已排序情況下能正確找出 median。

Case 2：反向排序輸入 (Reverse Sorted Case)

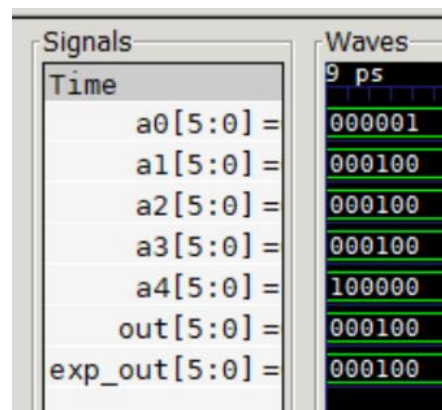
Input： a0 = 100000，a1 = 010000，a2 = 001000，a3 = 000100，a4 = 000010



此案例為完全反向排序情況，屬於排序電路中較具代表性的極端輸入之一，雖然資料順序與前例相反，但排序後的中間值仍應為 001000；模擬結果顯示輸出正確為 out = 001000，代表電路在反序排列下仍能正確完成 median 判斷。

Case 3：集中分布 (Clustered Values Case)

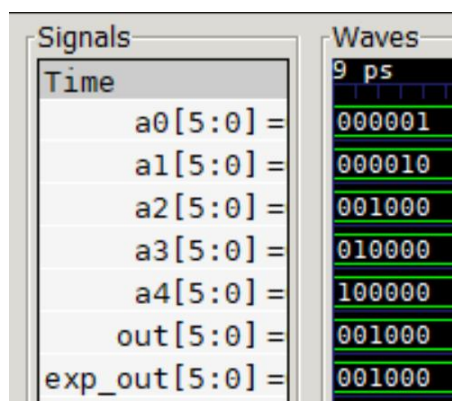
Input： a0 = 000001，a1 = 000100，a2 = 000100，a3 = 000100，a4 = 100000



此案例為多數輸入集中於相同數值的情況，其中 000100 出現三次，其餘為一個較小值與一個較大值，由於重複值位於中間區域，因此中位數應為 000100；模擬結果顯示輸出為 out = 000100，表示電路能正確處理重複值分布情況。

Case 4：邊界分布 (Boundary Spread Case)

Input： a0 = 000001，a1 = 000010，a2 = 001000，a3 = 010000，a4 = 100000



此案例橫跨較小值與較大值，輸入分布較廣，可用來驗證電路在跨區比較下的正確性；排序後位於中間位置的值為 001000，因此預期輸出應為該值；模擬結果顯示 out = 001000，證明電路在邊界分布情況下仍可正確找出中位數。

下圖：Median circuit 之 testbench 自動驗證結果

```
=====
Median Testbench Summary
Total cases : 7780
Pass cases   : 7780
Fail cases   : 0
=====
=====
          🐼 Simulation Finish ! 🐼 
=====
*****
**                                                    **
**                Congratulations !!                  **
**                All test passed!!                    **
**                                                    **
**                                                    **
*****
                                     .##. ....
               g#hghgghggh'ggg^g;
           gg      |: ddrv'ddpg|
         gh        ^hgghjdgfif
       ff              dg'h..
     fff-    gg      gg      jfgf'
  .dh-   fg    gh    ga    jfhg
    .gh.             .ghg
      fh            ..hh...ghghghghgh
=====
Simulation complete via $finish(1) at time 38905 NS + 0
./tb median.v:295                                $finish;
```

五、問題討論與反思

在這次設計過程中，我遇到最主要的問題，是第二種投票機（Voting M2）在驗證時無法直接沿用第一種（M1）的 testbench，一開始我認為是程式實作上出了問題，但實際檢查後發現，問題來自兩種架構在平手（tie）情況下的判斷規則不同，導致 comparator tree 在選擇勝者時方向不一致，因此即使邏輯本身正確，最終輸出仍會與 golden model 不符，進而產生大量 fail，後來我透過統一使用同種 tie-breaking rule 的思路去想，並重新調整 testbench 的驗證邏輯，才解決這個問題，這讓我實際理解到硬體設計中規格一致性比單一模組正確更重要，若在設計初期沒有定義清楚，後續驗證會變得非常困難。

另外，在 median circuit 的設計與分析中，我也發現自己一開始過於著重功能正確，而忽略了電路結構對效能的影響，例如在 critical path 與邏輯層數估算時，若只用模組數量去看，會低估實際延遲，因此後來我改用等效 NAND 開來估算邏輯深度，並搭配 Timing Analyzer 的結果進行修正，才讓分析更貼近實際情況，這個過程讓我理解到，設計不能只停留在功能可運作就好，尚需進一步考慮資料流走向、模組串接方式，以及這些因素如何影響延遲與效能。

綜合這些經驗，我認為對未來設計最大的影響，是在一開始就要同步思考規格、架構與驗證方式，而不是等電路寫完才來修正，那將會花費大量不必要的時間，未來在設計時，我會選擇先明確定義所有邊界條件（例如 tie 規則），並確保不同架構在同一規格下是可比較的，同時在規劃電路時，也會提前評估 critical path 的可能來源，避免後期才發現延遲過長而難以修改，此外，在 testbench 的設計上，也會優先建立清楚且對應規格的 golden model，讓驗證可以更有效率且有一致性，這些都是這次作業中實際遇過問題後，對未來設計流程最直接且重要的想法改變。